ORCHARD DOCUMENT 3

Network, Tunnels, Protocols, Cloud Boundary, and Network Security

Strict agent research and engineering scaffold aligned with Documents 1 and 2

Purpose: This addendum closes the networking gap. It defines what agents must research and document before Orchard's networking layer, tunnel boundary, cloud integration path, and protocol model are designed. It is written for local AI coding agents, infrastructure reviewers, and maintainers who must avoid casual network side effects.

Operating rule: No agent may implement networking, tunnels, proxying, DNS, packet filtering, or cloud exposure without completing the research tasks, safety checks, and architecture records in this document.

Document Map

1. Network Scope and Missing Research Correction

Documents 1 and 2 covered the core control-plane direction, Apple silicon optimization, runtime boundaries, security, and AI acceleration. Networking still needs a dedicated document because it is the area where local developer tools most often become unreliable: ports collide, service names fail to resolve, tunnels expose private services by accident, VPN clients interfere with routing, DNS becomes inconsistent, packet filters create invisible state, and cloud connectors quietly bypass local trust assumptions.

The networking layer must be treated as a first-class control-plane subsystem. It must have desired state, actual state, validation, diffing, apply planning, rollback behavior, ownership markings, cleanup, diagnostics, and policy events. Network state may not be treated as incidental shell output.

The first supported network capability set should prove local reliability before cloud reach. It should support local project isolation, deterministic port allocation, service discovery strategy, explicit ingress declarations, visible tunnel approvals, and security diagnostics. Cloud and tunnel features must be researched as connector boundaries before being promoted into supported behavior.

2. Apple Container Network Internals

Apple container’s technical overview says container integrates with vmnet, XPC, launchd, Keychain, and unified logging; the CLI communicates with container-apiserver and helper processes. The overview also states that container-apiserver launches container-core-images for image management, container-network-vmnet for virtual networking, and container-runtime-linux for each container. Orchard agents must map these components precisely before designing network state.

The research must distinguish public, documented behavior from internal helper implementation details. Orchard can observe, validate, and adapt to public behavior. It must not depend on private helper internals as stable APIs.

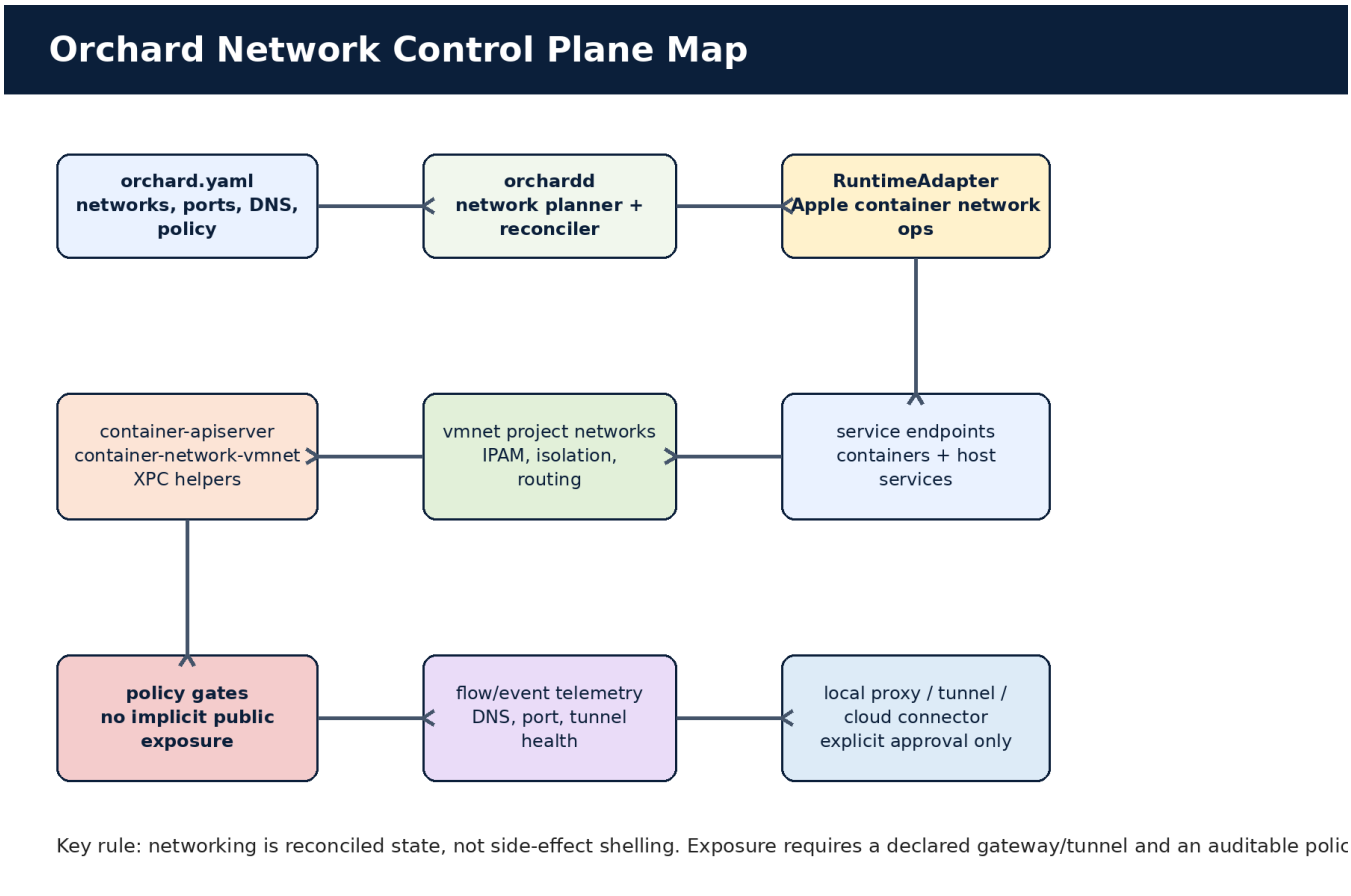

Figure 1. Orchard network control plane map. Networking is declared state, not incidental shell output.

Mandatory Apple Network Research Tasks

- Trace container network creation from CLI invocation through container-apiserver to container-network-vmnet.
- Document which container network commands exist on macOS 26 and which are unavailable on macOS 15.
- Measure project network creation latency, container attachment latency, teardown latency, and sleep/wake behavior.
- Find how Apple container assigns IP addresses, exposes container addresses, persists network metadata, and reports network errors.
- Determine how stable MAC addresses are represented, whether they can be configured, and whether they survive container recreation.
- Study vmnet interaction with host VPN clients, iCloud Private Relay, packet filter rules, captive portals, and corporate endpoint security tools.
- Identify what network state can be safely reconciled and what must only be diagnosed.

Required Runtime Findings

Topic	Required answer
Network object model	What is the durable identity of an Apple container network, and how does Orchard map it to a project?
Address allocation	Where are subnet, gateway, and endpoint addresses produced and observable?
Service discovery	Does Apple container provide any service-name resolution primitive, or must Orchard own discovery?
Network isolation	What isolation is guaranteed between projects, containers,

	and host services?
Host reachability	How do containers reach the host, and how does the host reach containers?
Cleanup	Which stale network resources survive failed apply, daemon crash, or sleep/wake?
Permissions	Which network operations trigger macOS prompts, require privileges, or interact with packet filtering?

3. Protocol Taxonomy and Service Communication Model

Orchard's networking design must start with protocol intent. A port number alone is not enough. A service endpoint should declare protocol, direction, exposure scope, TLS expectation, health check behavior, and whether it is reachable only inside the project, from localhost, through a tunnel, or from a remote cloud boundary.

Layer	Examples	Orchard implication
L3	IPv4, IPv6, CIDR, routing	Project network planning, subnet collision detection, host VPN conflicts, diagnostic routes.
L4	TCP, UDP, SCTP	Port allocation, local listeners, raw service reachability, health probes, policy rules.
L7	HTTP/1.1, HTTP/2, HTTP/3, gRPC, WebSocket	Gateway routing, protocol-aware health checks, headers, TLS, mTLS, audit metadata.
Name resolution	DNS, /etc/hosts, service aliases	Service discovery strategy and deterministic local names.
Security	TLS, mTLS, SSH, WireGuard, identity-aware tunnels	Explicit trust boundary and credential lifecycle.

Protocol declarations must be strict enough to support diagnostics. If a service says protocol: http, Orchard can perform HTTP health checks and route through an HTTP gateway. If it says tcp, Orchard should not assume HTTP semantics. If it says grpc, the gateway and health checks must support HTTP/2 behavior or fail validation.

```
services:
  api:
    image: ghcr.io/acme/api:latest
    endpoints:
      - name: http
        port: 3000
        protocol: http
        scope: project
        health:
          path: /health

  postgres:
    image: postgres:16
    endpoints:
      - name: sql
        port: 5432
        protocol: tcp
        scope: project
```

4. Project Networks, Service Discovery, and DNS

The baseline local model should be one isolated project network per Orchard project when supported by the runtime. Service-to-service communication should use stable names rather than hard-coded IP addresses. Docker Compose demonstrates the developer-experience expectation: a project gets a default network, services join it, and containers can discover each other by service name. Orchard must study whether Apple container can support the same user outcome without copying Docker's internals.

Discovery Options

Option	Pros	Risks	Research decision
Runtime-provided DNS	Smallest Orchard surface if Apple provides it	May not exist or may not be stable enough	Verify first.
Managed /etc/hosts injection	Simple for small static stacks	Poor with replicas, dynamic IPs, restarts	Possible limited fallback.
Local DNS resolver	Strong service-name model	macOS resolver integration, permissions, VPN conflicts	Research carefully.
Local reverse proxy	Good for HTTP/gRPC ingress	Does not solve arbitrary TCP service discovery	Good first gateway path.
Sidecar proxy	Protocol-aware and observable	Heavy for local dev; creates mesh complexity	Reject for first supported release unless proven necessary.

Project Network Contract

- Every managed container receives ownership metadata tying it to a project and network.
- Every project network has a deterministic name, stored desired state, observed state, and cleanup policy.
- Project networks must not collide with host VPN routes or existing private subnets without a warning and override.
- Name resolution must be deterministic and documented. IP stability cannot be promised unless the runtime supports it.
- A service with multiple replicas must define how clients choose an endpoint: direct list, local proxy, load-balanced gateway, or unsupported.

5. Ports, Ingress, Local Gateways, and Exposure Pipeline

Port publishing is a security decision. Orchard must not silently expose services beyond project scope. Localhost exposure, LAN exposure, tunnel exposure, and public exposure are separate states with separate approval, logging, and rollback behavior.

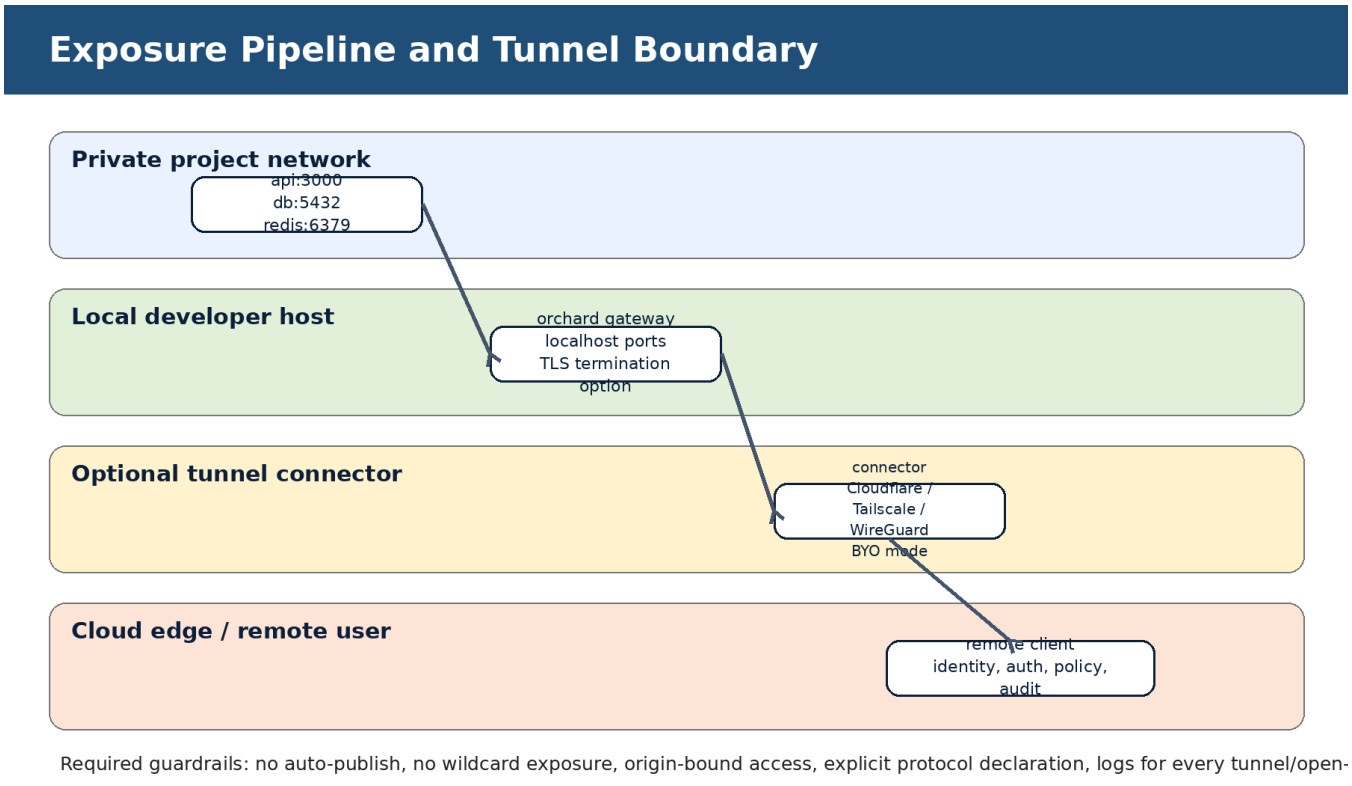

Figure 2. Exposure pipeline and tunnel boundary. Every step must be declared, validated, and auditable.

Exposure Scopes

Scope	Meaning	Default policy
project	Reachable only by services in the same Orchard project	Allowed when network is valid.
localhost	Reachable from the developer machine on 127.0.0.1 or localhost	Allowed only when declared.
lan	Reachable from local network interfaces	Blocked by default; requires explicit approval.
tunnel	Reachable through a declared tunnel connector	Blocked by default; requires connector policy and audit.
public	Internet-reachable through an edge service	Blocked until security review and explicit configuration.

```
gateways:
  local-api:
    scope: localhost
    routes:
      - service: api
        endpoint: http
        hostPort: 3000
        tls: false

  preview:
    scope: tunnel
    connector: cloudflare
    routes:
      - service: api
        endpoint: http
        hostname: preview.example.com
```

```
        auth: required
```

Port Allocation Rules

- Host ports must be reserved in Orchard state before containers are started.
- Port conflicts must fail during planning, not after partial runtime mutation.
- A service with replicas cannot bind all replicas to the same host port unless routed through a gateway.
- Ephemeral port assignments must be recorded and visible in status output.
- Release of a port must be tied to resource ownership and cleanup events.

6. Tunnel and Cloud Connector Research

Tunnel support has high product value and high security risk. Cloudflare Tunnel uses outbound-only connections from a local daemon to Cloudflare's network, avoiding direct inbound exposure to the origin. Tailscale uses WireGuard-based encrypted connectivity and identity-aware access between devices. WireGuard itself is a modern VPN protocol designed around simple public-key configuration. These are useful models for Orchard, but they must be treated as external connector systems with explicit policy boundaries.

Connector Boundary

Connector	Primary use	Orchard responsibility	Non-responsibility
Cloudflare Tunnel	Expose local HTTP/SSH/private services through Cloudflare edge	Generate/validate connector config, record exposure, show status	Own Cloudflare identity, account policy, or edge security.
Tailscale	Private mesh access to local services and dev machines	Detect tailnet context, route local service endpoints, document ACL assumptions	Replace Tailscale ACLs or identity control.
WireGuard	Low-level encrypted tunnel between peers	Research BYO tunnel support and endpoint policy	Operate arbitrary VPN lifecycle in first release.
SSH tunnel	Developer-controlled temporary forwarding	Document compatibility and risks	Make SSH tunneling implicit or invisible.

Tunnel Safety Requirements

- No tunnel is created by default.
- A tunnel route cannot be inferred from a port mapping; it must be declared.
- Every tunnel route must declare protocol, target service, auth expectation, hostname or route identity, and logging mode.
- Tunnel connectors must support dry-run rendering before activation.
- Tunnel status must show active connectors, target services, exposed hostnames, and last health check.
- Tunnel secrets must never be written into orchard.yaml or logs.
- Removing a tunnel must revoke or disable the corresponding local route where the connector supports it.

7. Cloud Infrastructure Boundary

Cloud integration should be shaped as exportable configuration and connector contracts, not as uncontrolled cloud automation. Orchard can help developers produce safe, repeatable local-to-cloud connectivity plans, but the first supported cloud boundary should favor generated configuration, explicit apply, and audit trails.

Cloud concern	Research requirement

Identity	OIDC, device identity, user identity, service token storage, and rotation model.
Secrets	Keychain usage locally; connector-specific secret stores externally.
DNS	Hosted zone changes, TTLs, local override behavior, wildcard risks.
TLS	Local certificates, tunnel-terminated TLS, edge-terminated TLS, mTLS for private services.
Load balancing	Local gateway versus cloud edge load balancer; no false HA claims for a laptop.
CI compatibility	How tests run in CI without Apple container, vmnet, or tunnel credentials.
IaC	Terraform/OpenTofu export as optional generated artifacts, not hidden magic.

Cloud features must never make a Mac laptop look like a production hosting platform. The cloud boundary exists for preview environments, secure team demos, private dev access, and test workflows. Availability claims must match the underlying device and connector behavior.

8. Network Security Model

Network security must be modeled in zones. Zone 0 is the macOS host. Zone 1 is the Apple container project network. Zone 2 is the exposure boundary: local gateway, published ports, tunnels, and cloud connectors. Every boundary crossing requires policy validation and an event record.

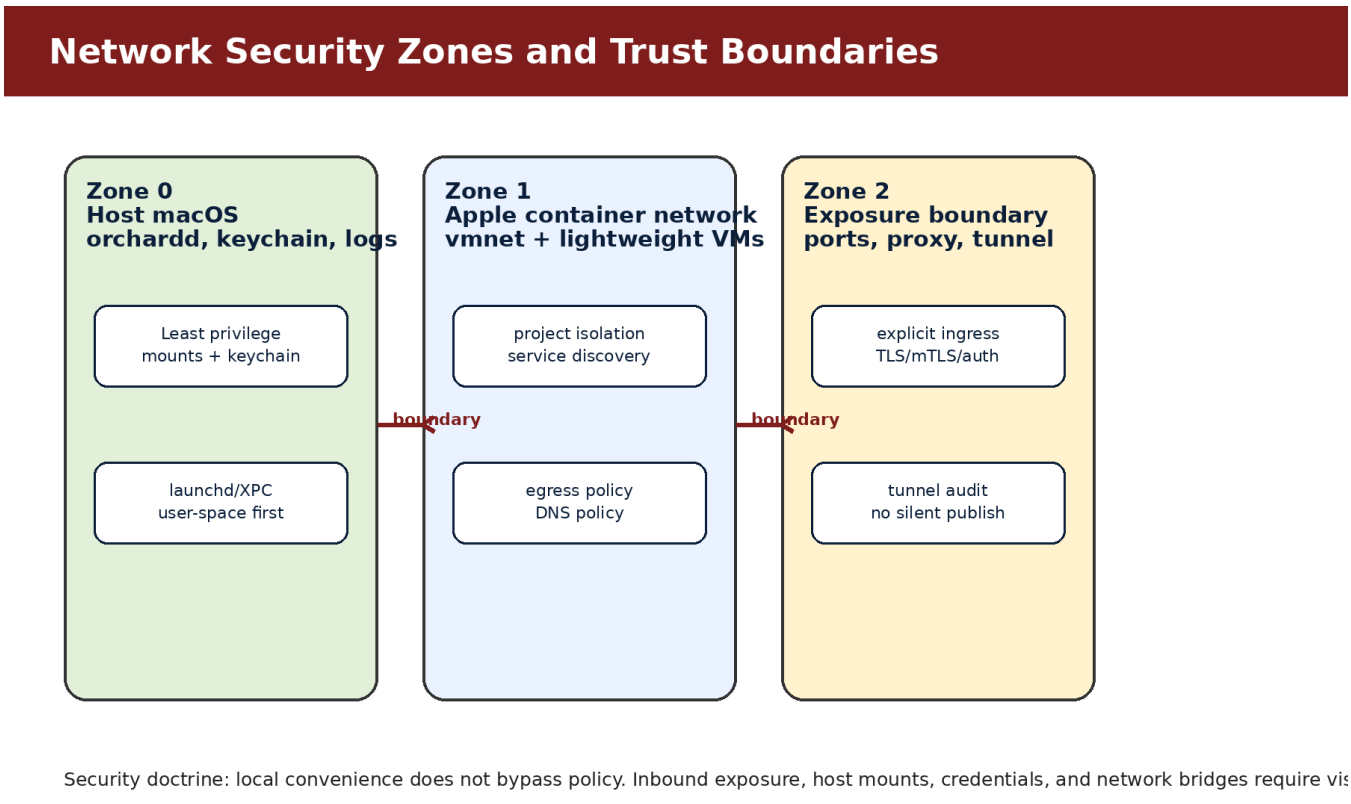

Figure 3. Network security zones and trust boundaries.

Required Network Policy Primitives

Primitive	Meaning
ingress.allow	Which sources may reach a service endpoint.
egress.allow	Which destinations a service may reach.
dns.allow	Which domains or resolvers are permitted.
exposure.scope	project, localhost, lan, tunnel, or public.
tls.mode	none, edge, local, mutual, or external.
auth.required	Whether external access requires identity enforcement.
audit.level	none, metadata, flow, or verbose diagnostics.

Kubernetes NetworkPolicy is useful as a conceptual reference because it defines application-centric rules for allowed communication, but Orchard must not assume Kubernetes policy semantics are available through Apple container. The first supported Orchard network policy should be smaller, local, explicit, and honest about enforcement gaps.

Network Threat Model

Threat	Required control
Accidental public exposure	No implicit tunnel/public exposure; explicit approval; event log.
Port collision or hijack	Preflight port reservation; post-apply validation; failure events.
DNS rebinding or spoofing	Document resolver path; pin service names; avoid trusting arbitrary hostnames.
SSRF from local stack	Egress policy research; metadata/IP block deny list for cloud contexts.
Secret leakage in tunnel config	Keychain or connector secret store; never plain orchard.yaml.
Overbroad host reachability	Least-privilege host service access; explicit host gateway declaration.
VPN or Private Relay route conflict	doctor checks for interfaces, routes, DNS, and packet filter state.
Stale packet filter/network rule	Ownership tags, cleanup dry-run, and safe garbage collection.

9. Observability, Diagnostics, and Failure Modes

Network bugs are often invisible. Orchard must provide diagnostics that explain what was declared, what was created, what is reachable, and where the path breaks. This requires event streams, network snapshots, endpoint health, DNS checks, route checks, and connector health checks.

Required Commands

```
orchard net status
orchard net inspect <project>
orchard net routes
orchard net dns
orchard net ports
orchard net doctor
orchard tunnel status
orchard tunnel render <connector>
orchard tunnel disable <name>
```

Failure mode	Agent must test
Container cannot resolve service name	DNS/service discovery diagnostics and fallback reporting.
Container can resolve but cannot connect	Port, route, policy, process-listener, and protocol checks.

Host port already used	Preflight detection and non-destructive failure.
VPN route conflict	doctor warning with affected CIDR and interface.
Tunnel connector down	Status command shows connector health and last error.
Sleep/wake breakage	Post-wake revalidation of networks, gateways, and tunnels.
Failed cleanup	Orphan list and manual recovery instructions.

10. Repository Additions and Documentation Structure

Document 3 adds a networking track to the repo. These files become part of the source of truth and must be written before networking code is considered ready for supported release.

```
docs/
  architecture/
    networking-overview.md
    apple-container-network-map.md
    service-discovery.md
    local-gateway.md
    tunnel-boundary.md
    cloud-connector-boundary.md
    network-security-model.md
    network-observability.md

  reference/
    network-manifest.md
    endpoint-scopes.md
    network-policy.md
    tunnel-connectors.md
    port-allocation.md
    network-error-codes.md
    doctor-network-checks.md

  design/
    adr-0013-networking-is-declared-state.md
    adr-0014-one-project-network-per-project.md
    adr-0015-no-implicit-public-exposure.md
    adr-0016-tunnel-connectors-are-external-boundaries.md
    adr-0017-protocol-aware-endpoints.md
    adr-0018-network-policy-before-cloud-exposure.md
```

Source layout additions:

```
Sources/
  OrchardNetworking/
    NetworkModel.swift
    NetworkPlanner.swift
    PortAllocator.swift
    ServiceDiscovery.swift
    LocalGateway.swift
    TunnelConnector.swift
    NetworkPolicy.swift
    NetworkDiagnostics.swift

Tests/
  OrchardNetworkingTests/
  OrchardNetworkPolicyTests/
  OrchardTunnelConnectorTests/
  OrchardNetworkIntegrationTests/
```

11. Strict Prompt for Local Agents

Use the following prompt for a local agent after it has Documents 1, 2, and 3 loaded into context.

```
You are working on Orchard, a Swift-native control plane for Apple container workloads. Do not
write networking implementation code yet.

Your task is to complete the networking research stage and produce source-backed docs.

Read Document 1, Document 2, and Document 3. Then study:
- apple/container technical overview and network commands
- apple/containerization networking-related sources
- Kubernetes cluster networking and NetworkPolicy docs
- CNI specification, especially ADD, DEL, CHECK, GC, and plugin/result model
- Docker and Compose networking docs for developer expectations
- Envoy docs for L7 proxy/gateway behavior
- Cloudflare Tunnel, Tailscale, and WireGuard docs for tunnel models
- macOS vmnet, packet filter, launchd, Keychain, and network permission behavior where official
docs exist

Deliver these files before coding:
1. docs/architecture/apple-container-network-map.md
2. docs/architecture/service-discovery.md
3. docs/architecture/tunnel-boundary.md
4. docs/architecture/network-security-model.md
5. docs/reference/network-manifest.md
6. docs/reference/doctor-network-checks.md
7. docs/design/adr-0013-networking-is-declared-state.md
8. docs/design/adr-0015-no-implicit-public-exposure.md
9. docs/design/adr-0016-tunnel-connectors-are-external-boundaries.md

Each document must include:
- sources read
- decisions made
- rejected alternatives
- risks
- tests required
- rollback/cleanup behavior
- unsupported behavior

Stop after documentation and review notes. Do not create tunnel code, DNS code, packet filter
code, or cloud automation without explicit approval.
```

12. Acceptance Gates and Open Questions

Acceptance Gates

- A reviewer can understand how a packet travels from service A to service B inside an Orchard project.
- A reviewer can understand how a localhost request reaches a container endpoint.
- A reviewer can understand how a tunnel route is declared, rendered, activated, audited, disabled, and cleaned up.
- No public or LAN exposure can happen without manifest declaration and visible approval.
- Network cleanup has dry-run output and ownership checks.
- The docs explain which network behavior comes from Apple container, which behavior Orchard owns, and which behavior remains unsupported.

- The test plan covers port conflicts, DNS failure, runtime network failure, VPN route conflict, tunnel failure, daemon restart, and laptop sleep/wake.

Open Questions

1. Can Apple container provide stable enough network metadata for reliable project-level reconciliation?

2. Can Orchard create one project network per project on macOS 26 without privilege escalation?

3. What service-discovery primitive is safest: runtime DNS, managed hosts files, local DNS, or local gateway?

4. How should service discovery work with replicas?

5. What happens to network state after laptop sleep and wake?

6. How do VPN clients, iCloud Private Relay, and enterprise packet filters alter vmnet behavior?

7. Can tunnel connectors be managed as generated config only, rather than runtime-owned processes?

8. What is the minimum tunnel support that is useful without becoming cloud automation?

9. How should Orchard represent ingress and egress policy if enforcement is partial?

10. Which network operations must require confirmation in interactive mode and block in noninteractive mode?

11. What cloud connector credentials can safely live in Keychain, and what must stay outside Orchard?

12. What network telemetry can be collected without privacy problems or excessive battery usage?

13. Source Basis

This document was created as an addendum to the Orchard agent manual and the security/Apple silicon acceleration addendum. It uses the following source families as anchors. Agents must re-check these sources during implementation because Apple container, cloud tunnel tooling, and networking APIs can change quickly.

- **Apple container technical overview:** https://github.com/apple/container/blob/main/docs/technical-overview.md - Runtime components, vmnet, XPC helpers, launchd, Keychain, macOS 15 networking limitations.
- **Apple Containerization repository:** https://github.com/apple/containerization - Swift package and native containerization surface for macOS.
- **Kubernetes Cluster Networking:** https://kubernetes.io/docs/concepts/cluster-administration/networking/ - Network problem taxonomy, Pod-to-Pod, Pod-to-Service, external-to-Service framing.
- **Kubernetes NetworkPolicy:** https://kubernetes.io/docs/concepts/services-networking/network-policies/ - Application-centric ingress/egress policy model and enforcement caveats.
- **CNI Specification:** https://github.com/containernetworking/cni/blob/main/SPEC.md - Plugin model, network configuration, ADD/DEL/CHECK/GC operations.
- **Docker networking overview:** https://docs.docker.com/engine/network/ - DNS, host port, subnet allocation, and local network expectations.

- **Docker Compose networking:** https://docs.docker.com/compose/how-tos/networking/ - Default project network and service-name discovery model.
- **Envoy documentation:** https://www.envoyproxy.io/docs/envoy/latest/intro/what_is_envoy - L7 proxy concepts, HTTP/gRPC routing, retries, circuit breaking, observability.
- **Cloudflare Tunnel docs:** https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/ - Outbound-only tunnel connector model and edge exposure considerations.
- **Tailscale concepts:** https://tailscale.com/docs/concepts/what-is-tailscale - Identity-aware WireGuard mesh and secure remote access model.
- **WireGuard:** https://www.wireguard.com/ - Modern VPN tunnel protocol and public-key connectivity model.